

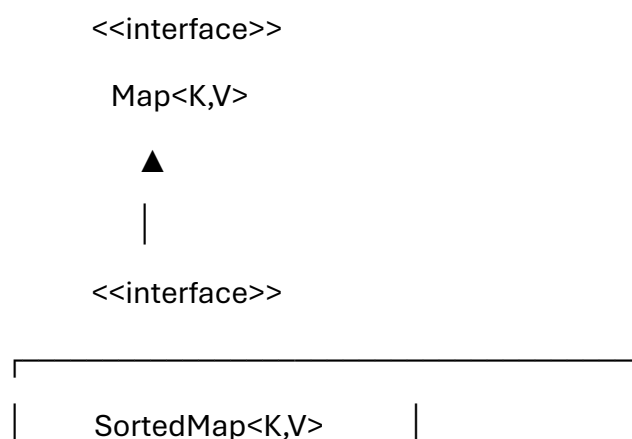
Aim

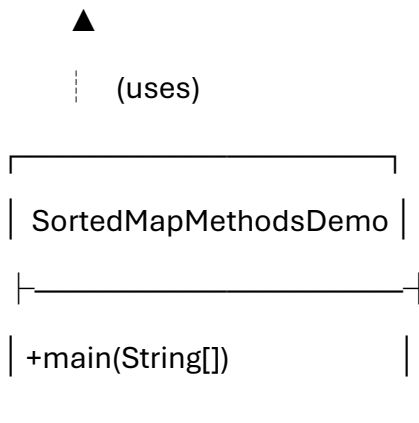
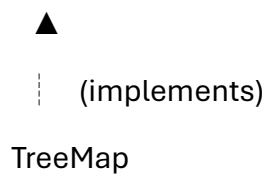
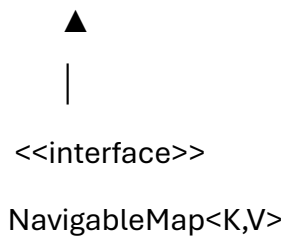
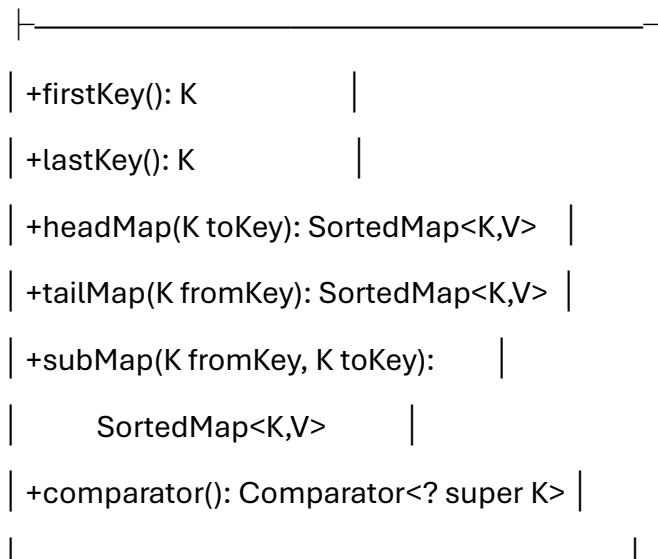
To write a Java program that demonstrates the SortedMap interface methods firstKey(), lastKey(), headMap(), tailMap(), subMap() and comparator() using a TreeMap.

Algorithm

1. Start.
2. Import the java.util package.
3. Create a SortedMap<Integer, String> object using TreeMap, mapping student ID to student name.
4. Add key-value pairs in unsorted order using put(), and display the map. The keys are automatically stored in ascending order.
5. Display the lowest key using firstKey() and the highest key using lastKey().
6. Display the sub-maps:
 - headMap(30) for the keys strictly less than 30.
 - tailMap(30) for the keys greater than or equal to 30.
 - subMap(20, 40) for the keys from 20 (inclusive) to 40 (exclusive).
7. Call comparator(). It returns null, since natural ordering is used.
8. Create a second TreeMap with Collections.reverseOrder() as its comparator and copy all entries into it using putAll().
9. Display the reverse-ordered map, its firstKey(), lastKey() and headMap(30), and check that comparator() is now not null.
10. Stop.

UML Diagram





Code:

```

package Observations;

import java.util.*;
  
```

```

public class SortedMapInterface {

    public static void main(String[] args) {

        SortedMap<Integer, String> sm = new TreeMap<>();

        // Insert in unsorted order; TreeMap keeps keys sorted
        sm.put(40, "Dinesh");
        sm.put(10, "Arun");
        sm.put(30, "Chitra");
        sm.put(20, "Bala");
        sm.put(50, "Eswar");

        System.out.println("SortedMap      : " + sm);

        // firstKey() and lastKey()
        System.out.println("firstKey()      : " + sm.firstKey());
        System.out.println("lastKey()       : " + sm.lastKey());

        // headMap(), tailMap(), subMap()
        System.out.println("headMap(30)      : " + sm.headMap(30));
        System.out.println("tailMap(30)      : " + sm.tailMap(30));
        System.out.println("subMap(20, 40)    : " + sm.subMap(20, 40));

        // comparator() - null means natural ordering
        System.out.println("comparator() is null : " + (sm.comparator() == null));

        // TreeMap with a custom comparator (descending order)
        SortedMap<Integer, String> rev = new TreeMap<>(Collections.reverseOrder());
        rev.putAll(sm);

        System.out.println("\nReverse-ordered map : " + rev);
    }
}

```

```

        System.out.println("firstKey()      : " + rev.firstKey());

        System.out.println("lastKey()       : " + rev.lastKey());

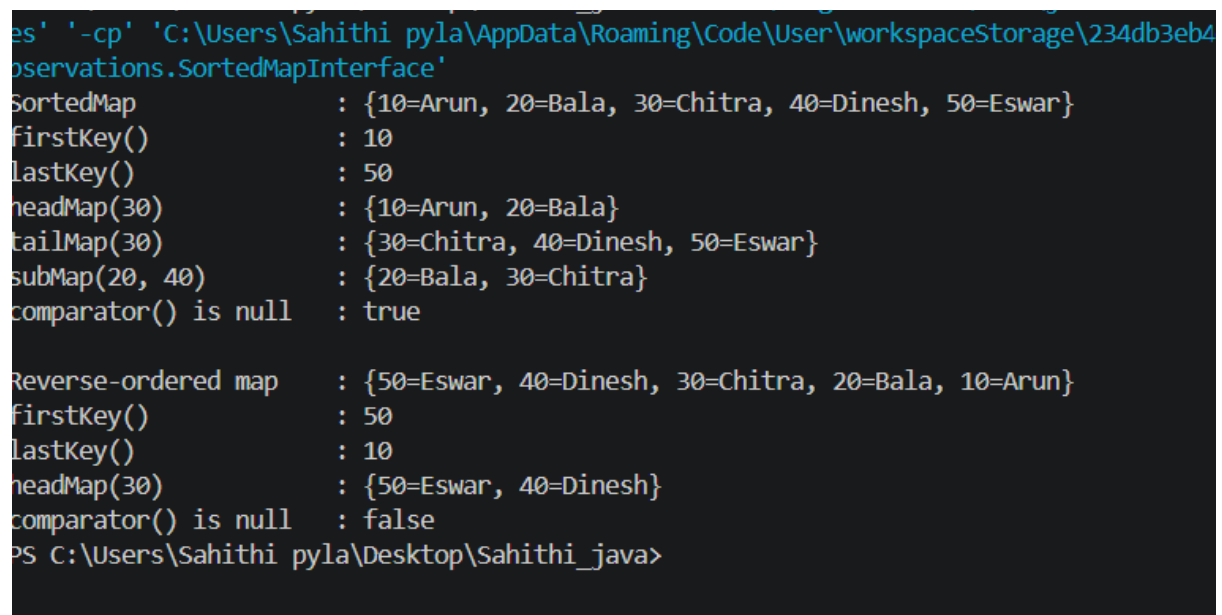
        System.out.println("headMap(30)     : " + rev.headMap(30));

        System.out.println("comparator() is null : " + (rev.comparator() == null));

    }
}

```

Output:



```

es' '-cp' 'C:\Users\Sahithi pyla\AppData\Roaming\Code\User\workspaceStorage\234db3eb4
observations.SortedMapInterface'
SortedMap          : {10=Arun, 20=Bala, 30=Chitra, 40=Dinesh, 50=Eswar}
firstKey()         : 10
lastKey()          : 50
headMap(30)        : {10=Arun, 20=Bala}
tailMap(30)        : {30=Chitra, 40=Dinesh, 50=Eswar}
subMap(20, 40)     : {20=Bala, 30=Chitra}
comparator() is null : true

Reverse-ordered map : {50=Eswar, 40=Dinesh, 30=Chitra, 20=Bala, 10=Arun}
firstKey()          : 50
lastKey()           : 10
headMap(30)         : {50=Eswar, 40=Dinesh}
comparator() is null : false
PS C:\Users\Sahithi pyla\Desktop\Sahithi_java>

```

Result:

The maps returned by headMap(), tailMap() and subMap() are views of the original map, not copies. Changes made through the view show up in the original map, and the other way round. If your lab needs independent copies, wrap them like new TreeMap<>(sm.headMap(30)).